Acadlore Transactions on AI and Machine Learning
https://www.acadlore.com/journals/ATAIML

Structural Verification and Drift Classification in Large Language Model-Compiled Rules: A Controlled Benchmark Study

Bailing Zhang*

Institute of Data Science and Intelligent Technology, NingboTech University, 315211 Ningbo, China

* Correspondence: Bailing Zhang (bailing.zhang1961@gmail.com)

Received: 04-30-2026 **Revised:** 06-08-2026 **Accepted:** 06-15-2026

Citation: B. L. Zhang, "Structural verification and drift classification in large language model-compiled rules: A controlled benchmark study," *Acadlore Trans. Mach. Learn.*, vol. 5, no. 2, pp. 167–179, 2026. https://doi.org/10.56578/ataiml050206.

Abstract: Large language models (LLMs) are increasingly used to turn natural-language knowledge into downstream executable rules, raising two lifecycle questions: whether a compiled rule faithfully preserves its source fragment's intended semantics, and whether later textual edits change rule behavior. We address these through a controlled benchmark based on inverted compilation: formal rules are generated programmatically, rendered into wiki-style fragments by an LLM, and reconstructed by an independent LLM pipeline, so the original rules provide automatic ground truth. The benchmark contains 2,000 rules across four business domains and 9,000 typed drift triples. For compilation verification, a slot-matched structural verifier reached 0.763 commit precision at a 32.1% commit rate, far exceeding a paraphrase-similarity baseline (0.729 precision, 2.4% commit rate). For drift classification, a slot-difference classifier was the only method that separated multiple impact categories, with F1 scores of 0.684 (condition), 0.601 (exception), and 0.419 (boundary), whereas token-level baselines collapsed to a coarse impactful-versus-cosmetic split. A complementary readiness-assessment experiment returned a negative result: surface-feature classifiers matched a majority-class baseline on synthesized fragments, indicating that readiness estimation needs authentic human-authored text or controlled degradations. Overall, slot-level structural analysis offers an effective signal for verifying and maintaining LLM-compiled rule systems, while exception extraction, cosmetic-edit discrimination, and the synthetic-to-real gap remain key limitations for future neuro-symbolic knowledge engineering.

Keywords: Large language models; Rule compilation; Structural verification; Drift classification; Neuro-symbolic artificial intelligence; Benchmark evaluation; Knowledge maintenance; Symbolic reasoning

1 Introduction

There is a recognizable pattern in applied artificial intelligence systems: a large language model (LLM) maintains a living document—a wiki, a policy repository, or a research knowledge base—while a separate downstream system requires executable formal rules derived from that document. Connecting these two layers raises a lifecycle question that neither the LLM community nor the rule compilation community has answered cleanly: how should a compiled rule be verified at the time of compilation, and how should it be maintained as the source document evolves?

The surface-level answer—recompile whenever anything changes—has three problems. First, not every edit to a knowledge fragment changes its logical content. Rephrasing, adding a citation, or reorganizing a paragraph may leave the underlying rule identical; treating these as recompilation triggers produces unnecessary work and review burden. Second, even when recompilation is warranted, the comparison must operate at the behavioral level rather than the syntactic level: two rules with different surface representations may be logically equivalent, while two rules that appear similar may diverge on edge-case inputs. Third, promotion of a compiled rule into the executable layer is a commitment—it should come with an explicit verification step and an audit trace, not a silent overwrite.

Evaluating any mechanism designed to address these problems has historically required human annotation: labelers must judge which compilations are faithful, which edits are impactful, and what the correct impact class is. This bottleneck has kept the problem at a small scale. Prior natural language-to-logic benchmarks such as LegalBench [1] and related regulatory translation tasks assume clean, static inputs and human-provided targets; they do not address the lifecycle question or scale beyond a few hundred labeled examples.

This study takes a different route. It can be observed that the compilation target—a formal rule in executable form—is itself a ground truth. A compiled rule can be evaluated by comparing its behavior with that of the original formal rule over a shared test input distribution. Similarly, a perturbation can be evaluated according to whether it induces a measurable change in the rule's behavior. If evaluation is driven by these automatic checks, the annotation bottleneck disappears and the benchmark can scale arbitrarily. This study operationalizes this observation as follows. After synthesizing formal rules procedurally from a library of domain templates, this study generates natural-language fragments from them using a LLM and measures whether a LLM-based pipeline can recover the original rule from those fragments. Drift is evaluated by applying typed programmatic perturbations to the formal rules, regenerating fragments from the perturbed versions and checking whether a drift classifier can recover the known perturbation type.

This study makes the following contributions by filling the gaps of prior work concerning natural language-to-logic translation and benchmark construction:

- This study proposes a fully automatic benchmark construction methodology based on inverted compilation, providing ground truth without human annotation and scaling to thousands of test cases. Unlike LegalBench [1], which assumes human-labeled formal targets, and the logic-language model [2], which evaluates a fixed pipeline on static inputs, the proposed framework covers the full source-to-rule lifecycle: compilation, verification, and drift detection.
- This study shows that a slot-matched structural verifier achieves a commit precision of 0.763 with a commit rate of 32.1%, compared with 0.729 precision and only a 2.4% commit rate for a token-Jaccard paraphrase baseline—a 13-fold improvement in recall at higher precision. The gap is explained by a structural vocabulary mismatch that slot-level comparison avoids.
- This study shows that a slot-difference classifier is the only method that achieves non-zero F1 on four of five drift impact classes, while both token-level baselines collapse to a binary impactful/cosmetic distinction. To the best of current knowledge, this is the first controlled study of typed drift classification on LLM-maintained rule fragments with automatic perturbation-based ground truth.
- This study reports a diagnostic negative result on promotion readiness assessment: surface-feature classifiers degenerate to the majority-class baseline on synthesized fragments, establishing that this sub-problem requires human-authored corpora with genuine quality variation and motivating two concrete future evaluation routes.

These results should not be interpreted as evidence of effectiveness on real-world, unconstrained wiki text; the gap between procedurally synthesized fragments and authentic human-written knowledge is a real limitation addressed in this study. The contribution is a controlled mechanism study that establishes a reproducible benchmark and characterizes the conditions under which slot-level analysis is effective and the conditions under which its limitations become apparent.

2 Related Work

Recent systems demonstrate that LLMs can maintain persistent, evolving knowledge stores. MemGPT [3] introduces virtual context management for LLM agents with long-term memory. Voyager [4] stores executable skills derived from natural language interaction. Reflexion [5] maintains verbal feedback traces across episodes. These systems stop at the narrative or skill representation layer; none defines a promotion interface to a downstream symbolic system or provides an evaluation protocol for source-to-rule lifecycle questions. The LLM wiki paradigm popularized by Karpathy [6] makes the same assumption: the wiki layer is the endpoint, not a source for downstream formal reasoning. Logic-language model [2] and related systems translate natural-language problems into symbolic solver inputs. LegalBench [1] provides a labeled benchmark for evaluating legal reasoning, including some natural language-to-logic subtasks. These systems assume clean, static, scoped inputs and human-provided ground truth for the compilation target. They do not address lifecycle questions—readiness gating, structural verification, or drift classification—and their evaluation protocols do not scale without annotation. More recent suites such as LogicBench [7] broaden systematic evaluation of logical reasoning, and surveys of generation faithfulness [8] motivate verifying a compiled rule rather than trusting it by default. The proposed approach borrows the compilation target from this line but replaces human annotation with automatic behavioral evaluation.

The proposed lifecycle state machine is recognizably related to classical truth maintenance systems [9, 10] and to the Alchourrón–Gärdenfors–Makinson belief revision theory [11]. These formalisms characterize how a knowledge base should update in response to new information and track justifications between beliefs. A closely related contemporary line is knowledge editing in LLMs, which modifies specific facts stored in model parameters [12–14]; in contrast, the present setting keeps knowledge as editable natural-language fragments and compiles them into an external symbolic layer, a neuro-symbolic stance surveyed by Marra et al. [15]. This study adds an operationalization over LLM-maintained text with a typed impact classification scheme; neither the truth maintenance system literature nor the Alchourrón–Gärdenfors–Makinson literature was developed for natural-language sources with LLM-compiled targets. The present work is also related to structured information extraction and business-rule mining. Those settings

often map text into schemas, fields, or rules, but the extracted objects are usually treated as final outputs for human inspection or downstream use. The focus of this study is narrower but more operational: the compiled rule is promoted into an executable layer, so the system must decide whether to commit the rule and later whether an edited source fragment changes its behavior. This lifecycle framing makes verification and drift classification central rather than incidental.

In program synthesis and neural theorem proving, it is standard to evaluate pipelines by inverting the generation direction: start from a formal object, generate the natural-language or input form from it, and measure whether a pipeline can recover the original [4]. The proposed benchmark construction methodology directly applies this pattern to rule compilation. To the best of current knowledge, this approach has not been systematically applied to LLM-maintained wiki-to-rule pipelines. Table 1 summarizes the key dimensions on which this study differs from the most closely related prior efforts. Compared with the logic-language model [2], this study introduces an evaluation methodology that works with any black-box compilation pipeline instead of a new translation architecture. Compared with LegalBench [1], the ground truth of this study is derived automatically from the compilation target rather than from human annotation, and the evaluation extends to cover the full lifecycle, including drift. Compared with the truth maintenance system, including the assumption-based truth maintenance system, and the Alchourrón–Gärdenfors–Makinson belief revision, this study operationalizes the theoretical update semantics in a concrete, measurable setting over LLM-generated text. The combination of lifecycle scope, annotation-free ground truth, and typed drift classification distinguishes this work from all three research streams.

Table 1. Comparison of this study with closely related work across key evaluation dimensions

Dimension	Logic-Language Model	LegalBench	Truth Maintenance System/The Alchourrón–Gärdenfors–Makinson Belief Revision	This Work
Annotation-free ground truth	–	–	–	√
Lifecycle scope (compile + drift)	–	–	√	√
Typed drift classification	–	–	–	√
Behavioral evaluation at scale	–	–	–	√
Negative result reported	–	–	–	√

Note: √ indicates that the dimension is explicitly addressed by the corresponding work, whereas – indicates that it is not addressed.

3 Problem Formulation

Let R be a formal rule in canonical form as follows:

$$R = \text{(head, body, guards, scope)}$$

where, head is the consequent, body is a conjunction of boundary conditions (numeric inequalities) and categorical constraints, guards is a set of exception clauses that disqualify the rule, and scope is a dictionary of domain metadata. A fragment f is a natural-language description of R produced by a generator function G: $R \mapsto f$.

Two rules, R_1 and R_2, are behaviorally equivalent on a test suite T if they produce identical outputs on every input:

$$\text{agree}\,(R_1, R_2, T) = \frac{1}{|T|} \sum_{x \in T} \mathbf{1}\left[R_1(x) = R_2(x)\right]$$

Behavioral equivalence is the primary correctness criterion throughout the study and can be evaluated automatically given T.

The slot parser S maps either a rule or a fragment to a structured representation:

$$S = (\mathcal{C}, \mathcal{E}, \mathcal{B}, \mathcal{K})$$

where, C is the set of condition slots (boundary and categorical), E is the set of exception slots, B is the set of numeric boundary expressions (a subset of $C \cup E$), and K is the set of scope key-value pairs. Each slot is represented as a canonical string (e.g., age $\geq$ 65.0, vehicle_class in {compact, sedan}).

Given two slot sets S_1 and S_2, the slot-level difference Δ(S1, S2) identifies which slot categories have changed:

$$\Delta = (\mathcal{C}_1 \backslash \mathcal{C}_2, \mathcal{C}_2 \backslash \mathcal{C}_1, \mathcal{E}_1 \backslash \mathcal{E}_2, \mathcal{E}_2 \backslash \mathcal{E}_1, \mathcal{B}_1 \backslash \mathcal{B}_2, \mathcal{B}_2 \backslash \mathcal{B}_1, \mathcal{K}_1 \backslash \mathcal{K}_2, \mathcal{K}_2 \backslash \mathcal{K}_1)$$

The difference drives both the structural verifier and the drift classifier.

Three research questions are formulated to evaluate readiness prediction, compilation faithfulness, and drift classification as follows:

Promotion readiness assessment (RQ1). Given a fragment f, this study investigates whether surface-level textual properties can predict successful compilation into a behaviorally faithful rule. This is treated as a diagnostic question rather than the main contribution, because synthesized fragments may not contain the quality variation found in human-authored wiki text.

Compilation faithfulness (RQ2). Given a fragment f generated from R, a pipeline P produces a predicted rule $\hat{R} = P(f)$. Whether $\hat{R}$ faithfully recovers R, as measured by agree $(R, \hat{R}, T)$, and whether a verification function $V(f, \hat{R})$ correctly decides whether to commit $\hat{R}$ are evaluated.

Drift classification (RQ3). Given a fragment pair (f_{before}, f_{after}), where f_{before} is generated from R and f_{after} from a perturbed rule R', whether a drift classifier $D(f_{before}, f_{after})$ correctly identifies the perturbation type without access to R or R' is assessed.

4 Methodology

The overall pipeline in Figure 1 describes the benchmark construction, the compilation and verification system, the perturbation and drift classification system, and a diagnostic readiness-assessment setting used to calibrate the synthetic-to-real gap. The compilation-faithfulness experiment evaluates compilation faithfulness by measuring behavioral agreement and slot-level precision between the compiled rule $\hat{R}$ and the ground-truth rule R under four verification strategies. The drift-classification experiment evaluates drift classification by applying typed perturbations to formal rules, regenerating fragments and comparing three classifiers on the resulting (before, after, label) triples.

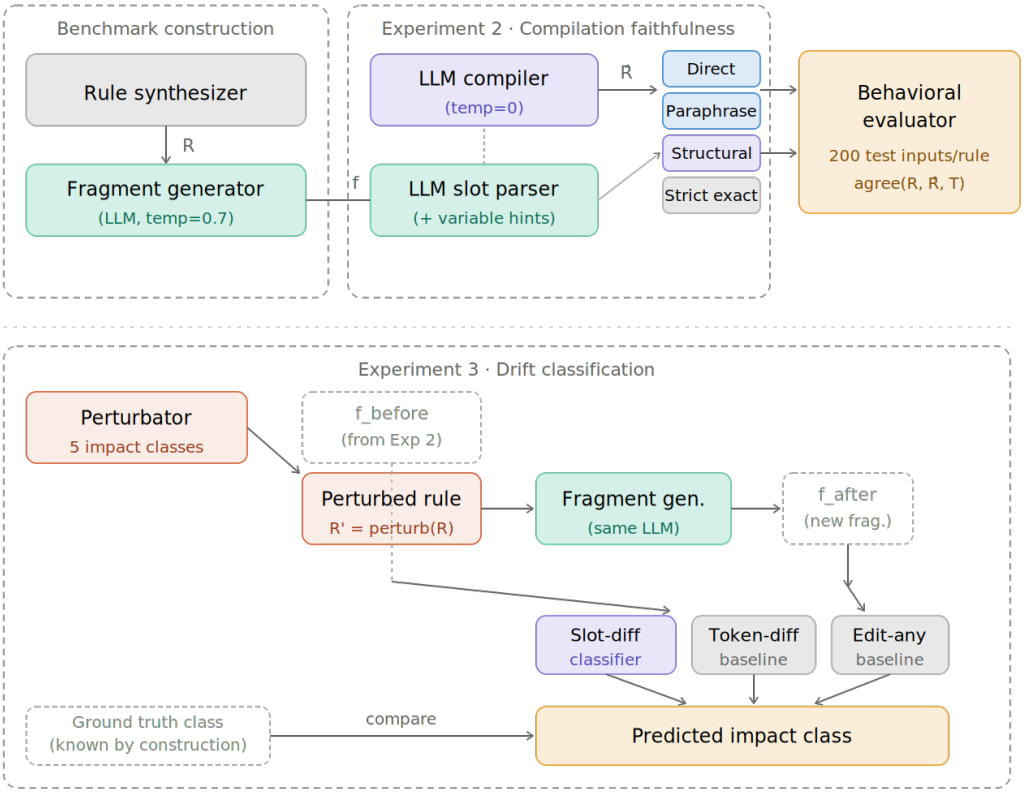

Figure 1. Illustrative overview of the GATED-PROMO benchmark pipeline

Note: LLM denotes large language model.

4.1 Benchmark Construction

Formal rules are synthesized programmatically from a library of four domain templates: insurance senior discount eligibility, high-risk insurance premium surcharge, personal loan approval, and Know Your Customer (KYC) compliance tier assignment. Each template specifies the variable schema (numeric ranges and categorical value sets), the number of boundary conditions and exception clauses, and the set of valid consequence values. Given

a template and a random seed, the synthesizer samples thresholds from the interior 60% of each variable's range (avoiding degenerate boundary rules), samples categorical subsets, and constructs a fully specified rule object. All generation is deterministic given the seed: rule i is generated with seed s0 + i, where s0 = 0 by default. Therefore, the entire benchmark is reproducible from a single integer. For the main experiments, 2000 rules distributed uniformly across the four templates are synthesized.

Table 2 summarizes the configuration of the four templates. Numeric variables are sampled uniformly from the interior 60% of the stated range, and categorical variables are sampled as uniform random subsets of size 1–4 from each template's value set.

Table 2. Template configurations for the four rule domains

Template	Key Variables (Range)	Number of Boundary Conditions per Rule	Number of Exception Clauses
Insurance senior discount	Age (18–90 years), policy tenure (0–40 years), claim count (0–15)	2–3	0–1
High-risk premium	Driving experience (0–50 years), accident count in previous 5 years (0–10), vehicle class	2–3	0
Lending approval	Credit score (300–850), debt-to-income ratio (0–1), loan amount, employment duration (months)	3	1
Know Your Customer (KYC) compliance tier assignment	Risk score (0–100), monthly transaction volume, account age (days), jurisdiction	2–3	0

An illustrative rule–fragment–compilation triple drawn from the benchmark is provided in Appendix A. For each rule R, a LLM (GLM-4-FlashX, temperature = 0.7) is used as a generator and prompted to produce a wiki-style natural-language paragraph describing the rule's conditions, consequence, and exceptions. The prompt instructs the generator to use natural, non-formulaic language while preserving all numeric thresholds and logical relationships exactly. One fragment is generated per rule in the main experiments. Behavioral evaluation requires concrete test inputs. For each rule, 200 test inputs are sampled by drawing each numeric variable uniformly from a range spanning one standard deviation beyond each threshold in the rule (ensuring dense coverage near decision boundaries) and by drawing categorical variables uniformly over their domain. Inputs are shared across all methods being compared so that behavioral agreement differences cannot be attributed to input distribution differences.

4.2 Compilation and Verification

The LLM compiler takes a fragment f and produces a predicted rule $\hat{R}$. It uses a structured JSON output prompt (GLM-4-FlashX, temperature = 0) with explicit schema specification and extraction rules addressing common failure modes: exception signal phrases (e.g., does not apply if and unless), percentage-to-fraction conversion for consequence values, ratio-scale normalization for debt-to-income thresholds, and lowercase normalization for categorical values. The compiler also handles malformed LLM output: duplicate JSON keys are merged by list concatenation, and bare numeric values in categorical slots are promoted to boundary conditions.

The following four verification strategies are implemented in parallel:

- Direct verification. Every successfully compiled rule is committed without verification. This is the no-gating baseline.
- Paraphrase verification. The predicted rule $\hat{R}$ is linearized into a canonical natural language string (conditions rendered as if clauses, consequences, and exceptions as unless clauses). The token Jaccard similarity between this paraphrase and the source fragment f is computed; the rule is committed if similarity exceeds a threshold (0.15 in the experiments of this study). This represents a surface-similarity verification baseline.
- Strict exact verification. The rule is committed only if its slot set is identical to the ground truth slot set, i.e., agree(R, $\hat{R}$, T) = 1.0 and $\Delta(S_R, S_{\hat{R}})$ is empty. This oracle strategy is not deployable (it requires R) but establishes a precision ceiling.
- Structural verification. The source fragment f is parsed into a slot set S_f by the LLM slot parser S, with the predicted rule's variable names supplied as hints to anchor the parser's output to known variable identifiers.

The slot difference $\Delta(S_f,\ S_{\hat{R}})$ is computed. The rule is committed if the difference contains no condition losses, no exception losses, and no boundary losses or gains that involve different variables on the two sides. Condition or exception gains (the pipeline adding extra constraints not in the source) are permitted.

The LLM slot parser converts a natural-language fragment into a slot set S using GLM-4-FlashX (temperature = 0). The parser prompt specifies the canonical string format for each slot type and accepts variable name hints to reduce naming inconsistencies. Raw parser output is normalized: numeric boundaries are parsed with the regex $|w + \cdot[><=!] + \cdot|d+$ and reformatted as variable op threshold:g to match the canonical format produced by the rule's $to\_slot\_set()$ method.

4.3 Perturbation and Drift Classification

For the drift-classification experiment, five typed perturbations are applied to each rule:

- Cosmetic perturbation. The rule structure is unchanged; a new fragment is generated from the same rule using a different random seed. The expected slot difference is empty.
- Boundary perturbation. One numeric threshold is shifted by $\pm 15\%$ of the variable's range, with direction sampled uniformly. The expected difference contains one condition or boundary modification with the same variable name but a different threshold.
- Condition perturbation. One condition boundary is removed from the rule body. The expected difference contains one condition loss.
- Exception perturbation. One exception boundary is removed. The expected difference contains one exception loss. This perturbation is only applied to rules that have at least one exception.
- Scope perturbation. One scope field value is replaced with a domain-alternative value (e.g., region: domestic $\rightarrow$ region: eu). The expected difference contains one scope modification.

Perturbation is deterministic given the rule and perturbation type, and the ground-truth impact class is known by construction.

Three methods are evaluated for assigning edits to predefined drift classes as follows:

- Slot-difference classifier. The canonical slot set of R_{before} (taken from the rule object directly, not re-parsed from text) is compared against the slot set parsed from fafter by the LLM slot parser. The difference is classified by priority: if an exception loss is detected, the edit is classified as EXCEPTION_CHANGE. Otherwise, if condition or boundary slots differ, the affected variable sets are compared. If the variable sets are identical, the edit is classified as BOUNDARY_CHANGE; otherwise, it is classified as CONDITION_CHANGE. If a scope modification is detected, the edit is classified as SCOPE_CHANGE. Finally, an empty difference or only minor symmetric noise ($\leq$4 slot changes involving identical variable sets) is classified as COSMETIC.
- Token-difference baseline. The symmetric token-set difference fraction between fbefore and fafter is computed. If it is above a threshold (0.15), the edit is classified as BOUNDARY_CHANGE (a generic impactful label); otherwise, it is classified as COSMETIC. This baseline cannot distinguish among impactful class types.
- Edit-any baseline. Every edit is classified as BOUNDARY_CHANGE. This achieves perfect recall on impactful classes at the cost of a 100% cosmetic false-positive rate.

It can be noted that the slot-difference classifier uses the canonical slot set for the pre-edit rule rather than re-parsing fbefore. This mirrors the realistic deployment setting in which a committed rule's slot structure is already known to the system; re-parsing the source fragment would introduce parser noise that inflates the apparent cosmetic false-positive rate.

5 Experimental Results

The following metrics were used in the experiments:

Behavioral agreement, agree(R, $\hat{R}$, T), is the fraction of the 200 sampled test inputs on which the ground-truth rule R and the compiled rule $\hat{R}$ produce identical outputs (1.0 = fully indistinguishable; 0.0 = never agree).

Commit threshold (τ = 0.95) defines a correct compilation. This study selects 0.95 rather than 1.0 because the strict-exact criterion is overly sensitive to non-integer threshold rounding: a rule with a threshold > 2.88 is logically equivalent to one with a threshold ≥ 3 on integer inputs but differs structurally. A 95% agreement rate on 200 inputs allows up to 10 edge-case discrepancies while remaining functionally stringent in practice.

Commit precision is the fraction of committed rules (those accepted by a verification strategy) whose behavioral agreement with the ground truth meets or exceeds τ. High precision means few incorrect rules pass through the gate.

Commit rate is the fraction of all fragments that result in a committed rule. A strategy with zero commit rate is trivially precise but operationally useless; the trade-off between precision and commit rate is the key comparison across verification strategies.

F1 score in the drift-classification experiment is the per-class positive-class F1 in a one-versus-rest scheme. Each of the five impact classes is evaluated separately, and the reported F1 is the harmonic mean of precision and recall for that class.

5.1 Compilation Faithfulness

After synthesizing 2,000 formal rules across four templates, one natural-language fragment per rule was generated, and all four verification strategies were evaluated. Behavioral agreement was measured on 200 test inputs per rule. The primary metrics were commit precision (fraction of committed rules with agreement ≥ 0.95 to the ground truth) and commit rate (fraction of fragments that result in a committed rule). In addition, slot-level precision, recall, and F1 were computed for the three main slot types (condition, exception, and boundary) by comparing the slots of the predicted rule against those of the ground-truth rule across all 2,000 rules, regardless of whether the rule was committed.

Table 3 reports per-strategy results. The direct strategy commits all 1,765 compilable fragments (88.2% of 2,000) with a precision of 0.729, meaning roughly 27% of committed rules have behavioral agreement below 0.95 with the ground truth. The paraphrase baseline commits only 48 fragments (2.4%), a severe reduction in recall attributable to the structural mismatch between the canonical paraphrase format and the free-form generator output: token Jaccard similarities are consistently below 0.15 except when the generator happens to use very similar vocabulary. The strict-exact strategy, which requires bit-perfect slot matches, commits 3 rules (0.2%) because synthesized non-integer thresholds are routinely rounded by the generator (e.g., 2.88 appears as "three or more"). The structural verifier commits 641 rules (32.1%) at a precision of 0.763. This represents a precision improvement of 3.4 percentage points over direct and paraphrase at a commit rate 13 times higher than paraphrase. The mean behavioral agreement of committed rules (0.965) exceeds that of the direct strategy (0.960) and paraphrase (0.957).

Table 3. Compilation-aithfulness experiment

Strategy	Committed Rules	Precision	Mean Behavioral Agreement	Commit Rate
Direct verification	1,765	0.729	0.960	0.882
Paraphrase verification	48	0.729	0.957	0.024
Strict exact verification	3	0.667	0.982	0.002
Structural verification (proposed)	641	0.763	0.965	0.321

Note: Mean behavioral agreement denotes the average behavioral agreement of committed rules. Because the strict-exact and paraphrase strategies commit very few rules, their precision estimates are subject to substantial uncertainty due to the small sample size.

Figure 2 shows commit precision and commit rate for each strategy. The structural verifier occupies a distinct position: higher precision than direct verification and paraphrase verification and a substantially higher commit rate than paraphrase verification and strict exact verification. The contrast between the paraphrase baseline (2.4% commit rate) and the structural verifier (32.1%) reflects a fundamental mismatch in what the two methods measure. The paraphrase baseline converts the compiled rule into a canonical template string and measures token-level overlap with the source fragment. This overlap is structurally low regardless of whether the compilation is correct, because the free-form LLM generator uses natural prose while the paraphrase template uses formal variable names and operators; the two texts have different vocabularies by design. The structural verifier avoids this problem entirely through comparisons at the slot level, where the representation is normalized. The failure type distribution confirms this interpretation: boundary weakening (815 cases) and condition loss (774 cases) are genuine compilation errors, not surface vocabulary mismatches, which is why the structural verifier can detect them while the paraphrase baseline cannot. As shown in the figure, the structural verifier achieves the highest precision while committing substantially more rules than the paraphrase or strict-exact baselines.

Table 4 and Figure 3 report slot-level precision, recall, and F1 across all 2,000 rules. Condition and boundary slots are extracted with F1 of 0.818 and 0.806, respectively, indicating that the compiler reliably captures the main logical structure. Exception slots are substantially weaker (F1 = 0.649), with 411 false positives and 322 false negatives across 2,000 rules. In Table 4, the numbers of true positives, false positives, and false negatives reflect slot-string exact matches after canonical normalization.

As shown in Figure 3, exception slots are consistently the weakest across all three metrics. The failure type histogram for the structural verifier shows that boundary weakening (815 cases) and condition loss (774 cases) are the most common reasons for non-commitment, followed by exception loss (281) and scope broadening (202). Boundary weakening arises primarily from two sources: rounding of non-integer thresholds during fragment generation (e.g., > 2.88 becomes "three or more") and comparator direction errors (e.g., $\geq$ is rendered as $>$). These are inherent to the round-trip design and not correctable at the compiler prompt level without additional disambiguation information.

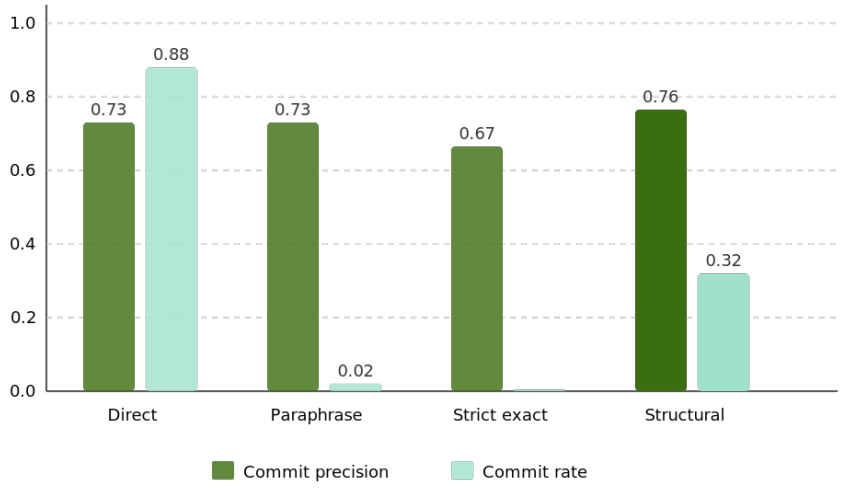

Figure 2. Commit precision and commit rate for the four verification strategies in the compilation-faithfulness experiment

Table 4. Compilation-faithfulness experiment

Slot Type	True Positives	False Positives	False Negatives	Precision	Recall	F1
Condition	4057	1068	739	0.792	0.846	0.818
Exception	677	411	322	0.622	0.678	0.649
Boundary	4105	1057	924	0.795	0.816	0.806

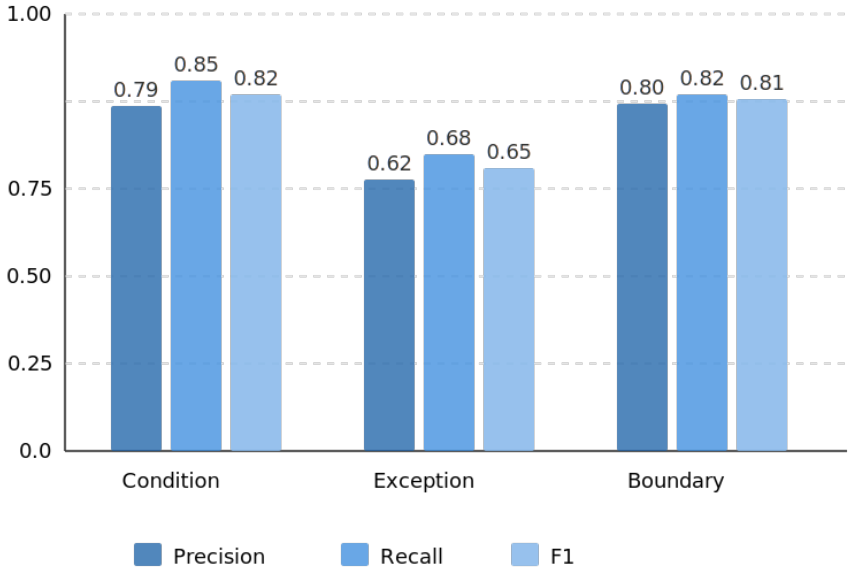

Figure 3. Slot-level precision, recall, and F1 for the three slot types in the compilation-faithfulness experiment

5.2 Drift Classification

Five perturbation types were applied to the 2,000 rules from the compilation-faithfulness experiment. Exception-change perturbations were only applicable to rules with at least one exception clause; these accounted for approximately 1,000 of the 9,000 total triples. Before-side fragments were taken directly from the compilation-faithfulness

experiment; after-side fragments were generated from the perturbed rules using GLM-4-FlashX (temperature = 0.7). The three classifiers were evaluated on all 9,000 triples. The primary metrics were overall accuracy, cosmetic false-positive rate (fraction of truly cosmetic edits classified as impactful), and impactful false-negative rate (fraction of impactful edits classified as cosmetic). In addition, per-class F1 was reported for the five impact classes.

Table 5 summarizes the per-method results. The edit-any and token-difference baselines achieve accuracies of 0.222 and 0.223, respectively, reflecting the class distribution in the test set. Given five drift classes and a roughly uniform distribution across the four impactful classes, a classifier that always predicts a single impactful class is expected to achieve an accuracy of approximately 22%. Both baselines have cosmetic false-positive rates near 1.0, meaning they trigger revalidation on essentially every edit regardless of whether the edit changes anything logically significant. Their impactful false-negative rates are correspondingly near zero. The slot-difference classifier achieves an overall accuracy of 0.482, more than twice that of both baselines. Its impactful false-negative rate (0.035) is low: it very rarely misclassifies an impactful edit as cosmetic. Its cosmetic false-positive rate (0.858) remains high, meaning it incorrectly flags most cosmetic edits as impactful; this limitation is discussed in Section 6.

Table 5. Drift-classification experiment results on 9,000 triples

Method	Accuracy	Cosmetic False Positive Rate	Impactful False Negative Rate
Edit-any	0.222	1.000	0.000
Token-difference	0.223	0.999	0.000
Slot-difference (proposed)	0.482	0.858	0.035

Note: Cosmetic false positive rate denotes the fraction of cosmetic edits incorrectly flagged as impactful. An impactful false negative rate denotes the fraction of impactful edits incorrectly classified as cosmetic.

The near-identical accuracy of the edit-any and token-difference baselines (0.222 vs. 0.223) reflects the fact that both methods effectively always predict impactful. The token-difference baseline crosses its 0.15 Jaccard threshold on the vast majority of perturbations because even cosmetic rewording changes enough tokens to exceed it. The slot-difference classifier's advantage is that it identifies the specific structural element changed, not merely something changed. The asymmetry across impact classes is instructive: exception change achieves near-perfect recall (false negatives = 3 across 9000 triples) because exception removal from the formal rule almost always causes a clearly detectable slot loss; scope change achieves low recall (R = 0.230) because scope fields are expressed with high lexical variability in natural language and the parser often normalizes different wordings to the same slot. This finding directly motivates more robust scope parsing as a target for future work.

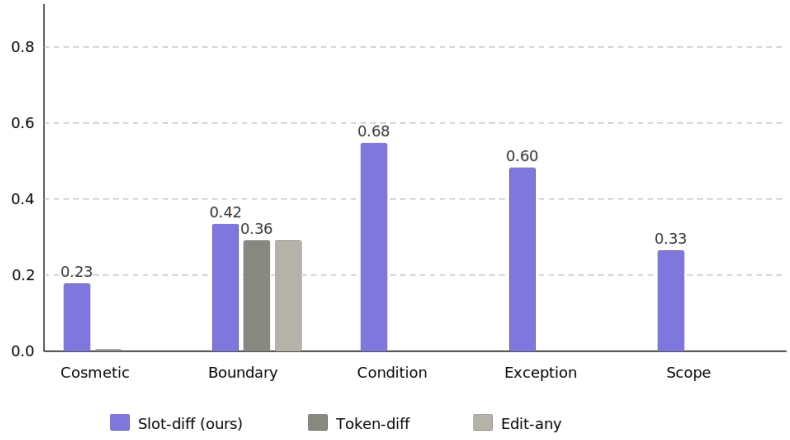

Figure 4. Per-class F1 for the three drift classification methods in the experiment

Figure 4 shows per-class F1 for the three methods. The slot-difference classifier is the only method with non-zero F1 on four of five impact classes. Condition change (F1 = 0.684) and exception change (F1 = 0.601) are the strongest classes, because their signals are relatively unambiguous in the slot diff: a condition disappears from the before slot set, or an exception disappears. Boundary change (F1 = 0.419) is harder because the boundary perturbation shifts a threshold value while keeping the variable name the same; the slot parser must produce exactly the shifted threshold on the after side for the difference to catch it. Scope change (F1 = 0.331) is the weakest impactful class: scope fields are expressed with high variability in natural language (e.g., for domestic customers, applicable to domestic policies,

and within the domestic market) and the parser often normalizes these to the same slot regardless of the semantic scope implied.

The token-difference baseline achieves non-zero F1 only on boundary change (F1 = 0.364), because boundary perturbations—shifting a numeric threshold by 15% of the variable range—reliably change the vocabulary of the fragment enough to cross the 0.15 token-difference threshold. Condition changes, scope changes, and exception changes are often expressed in ways that do not produce sufficient vocabulary change to cross this threshold. As shown in Figure 4, the slot-difference classifier achieves non-zero F1 on four of five classes; both baselines collapse to binary classification and cannot distinguish among impactful class types.

5.3 Diagnostic Study: Promotion Readiness Assessment

A fragment is labeled ready if the reference pipeline (direct compilation, no verification gating) recovers a rule with behavioral agreement ≥ 0.95 against the ground-truth rule on the 200-input test suite. Using the behavioral scores from the compilation-faithfulness experiment, 1,287 of 2,000 fragments received a ready label (64.3%). Five surface dimensions were extracted from each fragment text without any LLM calls: (i) boundary_explicit, a normalized count of numeric tokens saturating at five; (ii) consequence_stated, presence of consequence keywords (e.g., eligible, approved, surcharge, etc.); (iii) exception_stated, presence of exception keywords (e.g., however, unless, does not apply); (iv) no_hedging, the absence of vague qualifiers (e.g., approximately, may, roughly); and (v) no_temporal, the absence of temporal markers (e.g., as of, currently, since 2024).

The dataset was split into training, validation, and test sets using a 70/15/15 ratio, stratified by class label, and three classifiers were evaluated: a heuristic (fixed thresholds on word count, numeric count, and consequence keyword presence), a learned scalar (logistic regression over the five features, trained on the training split), and the proposed decomposed classifier (independent per-dimension thresholds combined by conjunction). The primary metric was binary F1 on the test split. As a secondary diagnostic, the information gain of each readiness dimension—defined as the reduction in binary F1 when that dimension is removed from the learned-scalar model—is reported in the text below rather than in Table 6; Table 6 lists only the aggregate per-classifier metrics (accuracy, precision, recall, F1, and true/false counts). Table 6 reports the results. All three classifiers degenerate to the majority-class baseline: the learned scalar achieves zero true negatives and zero false negatives, predicting ready for every test fragment, while the heuristic and decomposed classifiers commit one to two true negatives. Binary F1 for all methods (0.780–0.783) is explained entirely by the 64.3% class prior; no classifier meaningfully separates ready from not-ready. As shown in Table 6, all classifiers degenerate to the majority-class baseline (true negative ≈ 0).

Table 6. Diagnostic readiness study with promotion readiness classification on 302 test fragments

Method	Accuracy	Precision	Recall	F1	True Positive	True Negative
Heuristic classifier	0.639	0.641	0.995	0.780	193	0
Learned scalar	0.642	0.642	1.000	0.782	194	0
Decomposed classifier (proposed)	0.646	0.645	0.995	0.783	193	2

Information gain is 0.000 for each of the five readiness dimensions individually—boundary_explicit (0.000), consequence_stated (0.000), exception_stated (0.000), no_hedging (0.000), and no_temporal (0.000); removing any single feature has no effect on the learned scalar's F1, confirming that none of the five dimensions discriminates between ready and not-ready fragments. These per-dimension information-gain values are reported here in the text rather than in Table 6, which lists only the aggregate classification metrics. The failure is not a classifier design problem. LLM-generated fragments are uniformly structured: every fragment contains explicit numeric thresholds (the generator is instructed to preserve all thresholds exactly), consequence keywords, and exception clauses, and none uses hedging or temporal markers. The five feature dimensions have near-zero variance across the dataset. Therefore, neither a heuristic nor a learned model can separate the positive and negative class. The problem is a property of the benchmark input distribution, not of the readiness concept itself.

This result is direct experimental evidence for the synthetic-to-real gap discussed in Section 6: readiness assessment requires input text with genuine quality variation—incomplete thresholds, hedged conditions, domain jargon, and implicit logic—which synthesized fragments do not exhibit. This study reports this as a negative finding rather than omitting it, because it calibrates expectations for future work using this benchmark.

6 Discussion and Limitations

The precision advantage of structural verification (0.763 vs. 0.729 for direct) is modest in absolute terms but is not the primary argument for the approach. The more consequential finding is the 13 times higher improvement in commit rate over paraphrase verification at comparable precision. Paraphrase verification fails because the token Jaccard between a structured canonical paraphrase and a free-form wiki fragment is structurally low regardless of correctness:

the two texts have different vocabularies by design. Structural verification sidesteps this problem by comparing at the slot level, where the representation is normalized. The failure type histogram provides additional diagnostic value: the breakdown of non-committed rules by type (condition loss, boundary weakening, exception loss, and scope broadening) gives actionable information about where the compiler is failing, which paraphrase verification does not. The diagnostic readiness study shows that surface-feature readiness classifiers are not evaluable on synthesized fragments: all five dimensions are approximately constant across the dataset. Therefore, every classifier defaults to the majority class. This is the clearest demonstration of the synthetic-to-real gap in this study. The LLM generator is instructed to express all rule components explicitly and numerically, producing text that is uniformly compilable at the surface level regardless of what the underlying rule contains.

Real wiki text introduces at least three categories of quality variation that the synthetic benchmark cannot reproduce: (i) vague thresholds, where an author writes "a reasonably low credit score" rather than a precise number; (ii) implicit exceptions, where a disqualifying condition is mentioned in passing or omitted; and (iii) domain hedging, where regulatory uncertainty is expressed with qualifiers (e.g., typically, generally, and may) that make extraction ambiguous. Each would create genuine variance in the five readiness dimensions, enabling a classifier to discriminate. A meaningful future evaluation could proceed through one of two routes. The first is to source human-authored fragments from publicly available regulatory corpora—candidate sources include government compliance portals, clinical decision guidelines, and enterprise service-level agreements—and annotate compilability against the same behavioral-agreement criterion used in this study. The second is to apply controlled degradation to the existing synthetic fragments: selectively replacing numeric thresholds with vague quantifiers, omitting one exception clause, or inserting a hedging adverb, then measuring whether classifiers can detect the introduced ambiguity. Either route would preserve the automatic evaluation infrastructure from this study while providing the input variation that readiness classification requires.

The slot-difference classifier's cosmetic false-positive rate of 0.858 is the most important limitation. It means the system would trigger revalidation on 86% of edits that change only the wording, not the logical content of a fragment. The root cause is slot-parser instability: the same numeric threshold or categorical value can be expressed differently across two generations of the same rule, producing a non-empty slot difference even when the rules are identical. This is not an artifact of the perturbation design; it reflects a genuine limitation of LLM-based slot parsing on natural-language text with stylistic variation. Reducing this rate would require either more stable slot parsing (e.g., few-shot prompting with normalization examples, or structured parsing grammars) or a noise tolerance threshold in the difference comparison. Both the fragment generator and the pipeline LLM use GLM-4-FlashX. The pipeline may implicitly exploit phrasing patterns shared between the two roles, making recovery artificially easier than it would be with a fully independent generator. Our design calls for cross-backbone experiments using different models for generation and pipeline; this study defers this ablation to future work.

The fragments in this benchmark are generated by a LLM from formal rules. They are substantially more structured and complete than authentic human-written wiki text, which may hedge, omit details, use domain jargon, or express conditions implicitly. Performance on LegalBench subtasks with human-authored text would likely be lower than what this study reports; this research treats the synthetic setting as a controlled mechanism study rather than a deployment evaluation. Exception slots are consistently the weakest slot type across all experiments. Two distinct failure modes contribute: (i) the compiler sometimes reverses the comparator direction when extracting exception conditions from phrases like "if employment exceeds 380 months, this rule does not apply" (producing $\leq$ 380 rather than $\geq$380), and (ii) the slot parser sometimes classifies a condition as an exception or vice versa. The compiler prompt includes explicit rules for exception direction, but GLM-4-FlashX does not always follow them on edge-case phrasing. Upgrading to a stronger backbone model would likely reduce both error types.

7 Conclusion

This study describes a controlled benchmark methodology for two main mechanisms in the LLM-to-rule lifecycle: compilation faithfulness verification and drift impact classification. The methodology requires no human annotation and scales to thousands of test cases by inverting the compilation direction. Applied to 2,000 synthesized rules and 9,000 drift triples, it produces two positive findings: the structural verifier commits rules at a 32.1% commit rate with 3.4 percentage points higher precision than the direct baseline, and the slot-difference classifier is the only evaluated method that achieves non-zero F1 on four of five impact classes simultaneously. The diagnostic readiness study adds a useful negative result. Surface-feature classifiers fail to separate ready from not-ready fragments because the LLM-generated text is uniformly well formed, with near-zero variance on the readiness dimensions used. This does not invalidate readiness assessment as a problem; rather, it shows that the problem requires human-authored text or controlled degradation procedures before it can be evaluated meaningfully.

The results are specific to procedurally synthesized fragments in four business domains. Persistent exception-slot weakness, high cosmetic false-positive rate, and the readiness-assessment failure jointly indicate that real-world deployment requires stronger slot parsing and evaluation on human-authored source text. The benchmark construction

pipeline is reproducible, and the most direct follow-on work is to extend it to additional templates, different LLM backends, and real wiki or policy histories.

Data Availability

The benchmark construction scripts, synthesized rule specifications, generated fragments, perturbation labels, and aggregate evaluation outputs are available from the corresponding author upon reasonable request during review. The author intends to deposit the benchmark package in a public repository upon publication, subject to final removal of any access credentials or model-provider metadata.

Conflicts of Interest

The author declares no conflicts of interest.

Declaration on the Use of Generative AI and AI-assisted Technologies

Generative AI was used as part of the research pipeline, not as an author. GLM-4-FlashX was used during the experiments to generate wiki-style natural-language fragments, compile fragments into structured rule representations, and parse fragments into slot-level representations, as described in the Methodology section. The prompts, schemas, generated outputs, rule objects, and evaluation scripts were checked by the author through deterministic programmatic validation and aggregate error analysis. Generative AI was not used to fabricate experimental results, references, or conclusions. The author remains fully responsible for the manuscript content, experimental design, analysis, and interpretation.

References

[1] N. Guha, J. Nyarko, D. E. Ho, C. Ré, A. Chilton, A. Chohlas-Wood, A. Peters, B. Waldon, D. Rockmore, D. Zambrano *et al.*, "LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 44 123–44 279. https://doi.org/10.48550/arXiv.2308.11462

[2] L. Pan, A. Albalak, X. Wang, and W. Y. Wang, "Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, Singapore, 2023, pp. 3806–3824. https://doi.org/10.18653/v1/2023.findings-emnlp.248

[3] C. Packer, V. Fang, S. G. Patil, K. Oh, R. Agrawal, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv Preprint*, p. arXiv:2310.08560, 2023. https://doi.org/10.48550/arXiv.2310.08560

[4] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv Preprint*, p. arXiv:2305.16291, 2023. https://doi.org/10.48550/arXiv.2305.16291

[5] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Adv. Neural Inf. Process. Syst.*, vol. 36, p. arXiv:2303.11366, 2023. https://doi.org/10.48550/arXiv.2303.11366

[6] A. Karpathy, "LLM OS," https://karpathy.github.io, 2023.

[7] M. Parmar, N. Patel, N. Varshney, M. Nakamura, M. Luo, S. Mashetty, A. Mitra, and C. Baral, "LogicBench: Towards systematic evaluation of logical reasoning ability of large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 13 679–13 707. https://doi.org/10.18653/v1/2024.acl-long.739

[8] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023. https://doi.org/10.1145/3571730

[9] J. Doyle, "A truth maintenance system," *Artif. Intell.*, vol. 12, no. 3, pp. 231–272, 1979. https://doi.org/10.1016/0004-3702(79)90008-0

[10] J. de Kleer, "An assumption-based TMS," *Artif. Intell.*, vol. 28, no. 2, pp. 127–162, 1986. https://doi.org/10.1016/0004-3702(86)90080-9

[11] C. E. Alchourrón, P. Gärdenfors, and D. Makinson, "On the logic of theory change: Partial meet contraction and revision functions," *J. Symb. Log.*, vol. 50, no. 2, pp. 510–530, 1985. https://doi.org/10.2307/2274239

[12] N. De Cao, W. Aziz, and I. Titov, "Editing factual knowledge in language models," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 6491–6506. https://doi.org/10.18653/v1/2021.emnlp-main.522

[13] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT," *Adv. Neural Inf. Process. Syst.*, vol. 35, p. arXiv:2202.05262, 2022. https://doi.org/10.48550/arXiv.2202.05262

[14] Y. Yao, P. Wang, B. Tian, S. Cheng, Z. Li, S. Deng, H. Chen, and N. Zhang, "Editing large language models: Problems, methods, and opportunities," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore, 2023, pp. 10 222–10 240. https://doi.org/10.18653/v1/2023.emnlp-main.632

[15] G. Marra, S. Dumančić, R. Manhaeve, and L. De Raedt, "From statistical relational to neurosymbolic artificial intelligence: A survey," *Artif. Intell.*, vol. 328, p. 104062, 2024. https://doi.org/10.1016/j.artint.2023.104062

Appendix

Appendix A. Illustrative Rule–Fragment–Compilation Triple

The following example shows one complete data point from the benchmark, illustrating the correspondence between a formal rule, its LLM-generated fragment, and the compiled output.

Ground-truth rule (lending_approval_000002):

```
{"head": "loan_approved", "consequence_value": true, "conditions_boundary": [["
    credit_score", "<=", 686], ["employment_months", ">", 168], ["loan_amount", ">",
    282476]], "exceptions_boundary": [["debt_to_income", ">=", 0.8]], "scope": {"
    product": "personal_loan"}}
```

Generated fragment (GLM-4-FlashX, temperature = 0.7): "In the realm of personal loan approvals, a loan is deemed eligible for approval if the applicant's credit score is 686 or lower, they have been employed for more than 168 months, and the loan amount exceeds $282,476. However, if the debt-to-income ratio is 80% or higher, the loan application does not qualify for automatic approval under the criteria."

Compiled output (success = true, behavioral agreement = 0.97 on 200 test inputs): the pipeline correctly recovers all three condition slots and the exception slot. The debt-to-income threshold is normalized from the percentage representation (80%) to the ratio scale (0.8) by an explicit extraction rule in the compiler prompt, demonstrating how the compiler's normalization logic handles unit mismatches introduced during fragment generation.